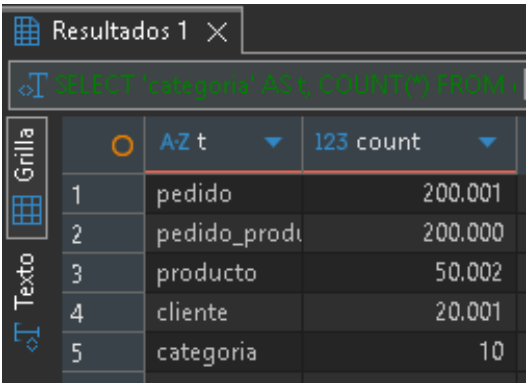


TP3 Semana 3 — Optimización con IA en Food Store

Fabri — UTN. Base **food_store_copia** (nunca toqué la dev). PostgreSQL + DBeaver en Manual, con backup antes como dice el protocolo.

1. Carga masiva: lo que me pasó de verdad

Arranqué con 1 categoría, 2 productos (Muzza 100 y Napolitana 120), 1 cliente y 1 pedido. Pedí el script con generate_series y cuando lo corrí DBeaver me tiró **No active connection** porque había abierto el .sql en <none>. Lo conecté a la copia y después me comí el **428C9: GENERATED ALWAYS** — no me dejaba meter ids a mano. Lo arreglé con **OVERRIDING SYSTEM VALUE** en los 4 INSERT y ahí sí entró todo. Después me quedó la transacción abortada (25P02) y la limpié con Revertir. Dejé 10 categorías (la original era **Pizzas**), 50002 productos, 20001 clientes, 200001 pedidos y 200000 detalles. Para no violar R1 puse precio_unitario=500 porque mis nuevos tienen lista 500-5000 (los viejos de 100/120 me lo hubieran roto). Para R2 usé NOW()-random. Cerré con COMMIT y los 5 ANALYZE.



Resultados 1 X

SELECT 'categoria' AS t, COUNT(*) FROM ...

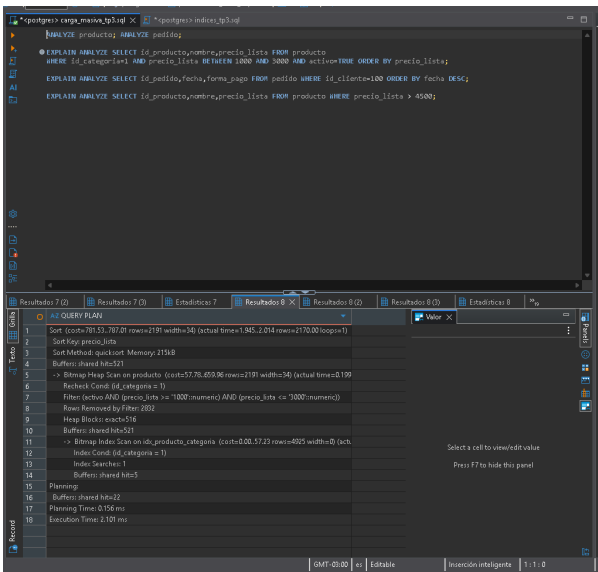
	AZ t	123 count
1	pedido	200.001
2	pedido_produ	200.000
3	producto	50.002
4	cliente	20.001
5	categoria	10

2. Las 3 lentas que medí

Medí antes con EXPLAIN ANALYZE. Q1 me dio ~2.1 ms con Bitmap sobre idx_producto_categoria + Sort, filtrando 2832 filas. Q2 0.05 ms con idx_pedido_cliente + Sort (eran solo 10 filas). Q3 la tuve que cambiar: la original usaba las PK y volaba (0.543 ms), así que armé una por precio >4500 que sí hizo Seq Scan de 50k y tardó 3.394 ms con 44466 descartadas.

Consulta	Antes	Qué le puse	Después	Mejora
Q1 cat 1 + precio 1000-3000 + orden	Bitmap idx_producto_categoria + Sort 2.101 ms	idx_producto_cat_precio_activo (cat,precio) WHERE activo	Bitmap con el nuevo + Sort 1.430 ms	1.46x, la acepté
Q2 cliente 100 ORDER BY fecha	Bitmap idx_pedido_cliente + Sort 0.050 ms	idx_pedido_cliente_fecha	Sigue viejo 0.067 ms	No funcionó
Q3 precio >4500	Seq Scan 3.394 ms, descartó 44466	idx_producto_precio	Bitmap Index Scan 1.636 ms	2.07x, la mejor

Q1 antes:



Explain Analyze results for Q1:

```
Explain Analyze SELECT id_producto,nombre,precio_lista FROM producto
WHERE id_categoria=1 AND precio_lista BETWEEN 1000 AND 5000 AND activo=TRUE ORDER BY precio_lista;

Explain Analyze SELECT id_pedido,fecha,forma_pago FROM pedido WHERE id_cliente=100 ORDER BY fecha DESC;

Explain Analyze SELECT id_producto,nombre,precio_lista FROM producto WHERE precio_lista > 4500;
```

Query Plan details:

- Sort (cost=761.53..767.01 rows=2191 width=34) (actual time=1.945..2.014 rows=2170 loops=1)
- Sort Key: precio_lista
- Sort Method: quicksort Memory: 215kB
- Buffers: shared hits=521
- Bitmap Heap Scan on producto (cost=52.76..809.96 rows=2191 width=34) (actual time=0.199..)
- Bitmap Index Cond: id_categoria = 1
- Filter: (activo AND (precio_lista >= 1000::numeric) AND (precio_lista <= 5000::numeric))
- Rows Removed by Filter: 2832
- Heap Block reads: 16
- Buffers: shared hits=521
- Bitmap Index Scan on idx_producto_categoria (cost=0.00..57.23 rows=4925 width=0) (act..)
- Index Cond: id_categoria = 1
- Index Searches: 1
- Buffers: shared hits=5
- Planning: 0.156 ms
- Execution Time: 2.101 ms

Q2 antes:

Auto | postgres | public@food_store_copa |

postgres=# \i cargo_maximo_bp.sql

```

ANALYZE producto; ANALYZE pedido;

-- EXPLAIN ANALYZE SELECT id_producto, nombre, precio_lista FROM producto
-- WHERE (id_categoria=1) AND precio_lista BETWEEN 1000 AND 5000 AND order_time ORDER BY precio_lista;
EXPLAIN ANALYZE SELECT id_pedido, fecha, forma_pago FROM pedido WHERE id_cliente=100 ORDER BY fecha DESC;
EXPLAIN ANALYZE SELECT id_producto, nombre, precio_lista FROM producto WHERE precio_lista > 4500;

```

Resultados 7 (Q) | Resultados 7 (Q) | Estadísticas 7 | Resultados 8 (Q) | Resultados 8 (Q) | Estadísticas 8 |

1 | HT QUERY PLAN

2 | Sort (cost=0.15..41.77 rows=10 width=20) (actual time=0.036..0.038 rows=10 loops=1)

3 | Sort Key: fecha DESC

4 | Sort Method: quicksort Memory: 25kB

5 | Buffers: shared hit=12

6 | -> Bitmap Heap Scan on pedido (cost=4.17..42.02 rows=10 width=20) (actual time=0.02)

7 | Recheck Cond: (id_cliente = 100)

8 | Heap Blocks: exact=10

9 | Buffers: shared hit=12

10 | -> Bitmap Index Scan on idx_pedido_cliente (cost=0.00..4.17 rows=10 width=0) (actual

11 | Index Cond: (id_cliente = 100)

12 | Index Searches: 1

13 | Buffers: shared hit=12

14 | Planning:

15 | Buffers: shared hit=15

16 | Planning Time: 0.116 ms

17 | Execution Time: 0.050 ms

GMT-03:00 | Editar | Inspección inteligente | 1:1:0

Q3 antes (Seq Scan):

Auto | postgres | public@food_store_copa |

postgres=# \i cargo_maximo_bp.sql

```

ANALYZE producto; ANALYZE pedido;

-- EXPLAIN ANALYZE SELECT id_producto, nombre, precio_lista FROM producto
-- WHERE (id_categoria=1) AND precio_lista BETWEEN 1000 AND 5000 AND order_time ORDER BY precio_lista;
EXPLAIN ANALYZE SELECT id_pedido, fecha, forma_pago FROM pedido WHERE id_cliente=100 ORDER BY fecha DESC;
EXPLAIN ANALYZE SELECT id_producto, nombre, precio_lista FROM producto WHERE precio_lista > 4500;

```

Resultados 7 (Q) | Resultados 7 (Q) | Estadísticas 7 | Resultados 8 (Q) | Resultados 8 (Q) | Estadísticas 8 |

1 | HT QUERY PLAN

2 | Seq Scan on producto (cost=0.00..1141.01 rows=5111 width=30) (actual time=0.013..3.349 rows=5176 loops=1)

3 | Filter: (precio_lista > 4500::numeric)

4 | Row Removed by Filter: 4666

5 | Buffers: shared hit=16

6 | Planning Time: 0.075 ms

7 | Execution Time: 3.394 ms

GMT-03:00 | Editar | Inspección inteligente | 1:1:0

Q1 después (usa índice nuevo, 1.430 ms):

Resultados 1 X

EXPLAIN ANALYZE SELECT id_producto, ne Enter a SQL expression to filter results (use Ctrl+Space)

Grilla

Texto

AZ QUERY PLAN

1 | Sort (cost=777.38..783.00 rows=2246 width=34) (actual time=1.257..1.339 rows=2170 loops=1)

2 | Sort Key: precio_lista

3 | Sort Method: quicksort Memory: 215kB

4 | Buffers: shared hit=533

5 | -> Bitmap Heap Scan on producto (cost=97.05..652.36 rows=2246 width=34) (actual time=0.148..0.785 rows=2

6 | Recheck Cond: ((id_categoria = 1) AND (precio_lista >= '1000'::numeric) AND (precio_lista <= '3000'::numer

7 | Heap Blocks: exact=514

8 | Buffers: shared hit=533

9 | -> Bitmap Index Scan on idx_producto_cat_precio_activo (cost=0.00..96.49 rows=2246 width=0) (actual tim

10 | Index Cond: ((id_categoria = 1) AND (precio_lista >= '1000'::numeric) AND (precio_lista <= '3000'::numer

11 | Index Searches: 1

12 | Buffers: shared hit=19

13 | Planning:

14 | Buffers: shared hit=3 dirtied=1

15 | Planning Time: 0.154 ms

16 | Execution Time: 1.430 ms

Q2 después (sigue viejo, 0.067 ms):

Resultados 1	
EXPLAIN ANALYZE SELECT id_pedido, fecha	
Enter a SQL expression to filter results (use Ctrl+Space)	
AZ QUERY PLAN	
1	Sort (cost=42.19..42.22 rows=10 width=20) (actual time=0.041..0.042 rows=10.00 loops=1)
2	Sort Key: fecha DESC
3	Sort Method: quicksort Memory: 25kB
4	Buffers: shared hit=12
5	-> Bitmap Heap Scan on pedido (cost=4.37..42.02 rows=10 width=20) (actual time=0.025..0.034 rows=10.00 loops=1)
6	Recheck Cond: (id_cliente = 100)
7	Heap Blocks: exact=10
8	Buffers: shared hit=12
9	-> Bitmap Index Scan on idx_pedido_cliente (cost=0.00..4.37 rows=10 width=0) (actual time=0.013..0.013 rows=10.00 loops=1)
10	Index Cond: (id_cliente = 100)
11	Index Searches: 1
12	Buffers: shared hit=2
13	Planning Time: 0.085 ms
14	Execution Time: 0.067 ms

Q3 después (Bitmap idx_producto_precio, 1.636 ms):

Resultados 1	
EXPLAIN ANALYZE SELECT id_producto, precio	
Enter a SQL expression to filter results (use Ctrl+Space)	
AZ QUERY PLAN	
1	Bitmap Heap Scan on producto (cost=106.44..690.43 rows=5439 width=34) (actual time=0.417..1.490 rows=5536 loops=1)
2	Recheck Cond: (precio_lista > '4500'::numeric)
3	Heap Blocks: exact=516
4	Buffers: shared hit=516 read=17
5	-> Bitmap Index Scan on idx_producto_precio (cost=0.00..105.08 rows=5439 width=0) (actual time=0.367..0.368 rows=5439 loops=1)
6	Index Cond: (precio_lista > '4500'::numeric)
7	Index Searches: 1
8	Buffers: shared read=17
9	Planning Time: 0.100 ms
10	Execution Time: 1.636 ms

Ojo: al principio medí sin haber creado los índices (solo ANALYZE) y me daban igual. Me fijé en pg_indexes, vi que seguían 4 y ahí los creé de verdad.

3. Lo que la IA me explicó mal

Le pasé el plan de Q3 con índice y me lo explicó prolijo pero con pifies clásicos. Lo contrasté línea por línea:

Lo que dijo la IA	¿Sí?	Lo que vi yo
cost 107..694 = tardó eso en ms	No	cost es estimado. Real es actual 0.417..1.490 + Exec 1.636
mejoró por el índice compuesto de Q1	No	El plan dice idx_producto_precio
ya no descarta filas	No	Antes 44466. Ahora Recheck + 516 bloques
hit 516 = disco	No	hit=RAM, read=disco. hit 516 read 17

4. Dos consultas con spec y EXCEPT

Spec1: por cada categoría activa, nombre + cantidad de productos activos, con 0, orden cant DESC. Una con JOIN+GROUP y otra con subconsulta. Las dos me dieron Pizzas 5002 y las 9 masivas en 5000. EXCEPT ida y vuelta 0.

categoria 1 X categoria 1 (2) X Estadísticas 1

SELECT c.nombre AS nombre_categoria, C

Enter a SQL expression to filter results (use Ctrl+Space)

Grilla

AZ nombre_categoria

123 cantidad

1	Pizzas	5.002
2	Categoría masiva 1	5.000
3	Categoría masiva 2	5.000
4	Categoría masiva 3	5.000
5	Categoría masiva 4	5.000
6	Categoría masiva 5	5.000
7	Categoría masiva 6	5.000
8	Categoría masiva 7	5.000
9	Categoría masiva 8	5.000
10	Categoría masiva 9	5.000

categoria 1 X categoria 1 (2) X Estadísticas 1

SELECT c.nombre AS nombre_categoria, C

Enter a SQL expression to filter results (use Ctrl+Space)

Grilla

AZ nombre_categoria

123 cantidad

1	Pizzas	5.002
2	Categoría masiva 1	5.000
3	Categoría masiva 2	5.000
4	Categoría masiva 3	5.000
5	Categoría masiva 4	5.000
6	Categoría masiva 5	5.000
7	Categoría masiva 6	5.000
8	Categoría masiva 7	5.000
9	Categoría masiva 8	5.000
10	Categoría masiva 9	5.000

Spec2: clientes con pedido TARJETA últimos 90 días. Una con EXISTS y otra con JOIN DISTINCT. Como cargué todo en EFECTIVO, con TARJETA dan 0 y 0 (igual vale). Con EFECTIVO también 0 de diferencia.

Resultados 2 X

(SELECT c.nombre, COUNT(p.id_producto

Enter a SQL expression to filter results (use Ctrl+Space)

Grilla

AZ nombre

123 count

5. Competencia + DUIA

Equipo	Estrategia	Antes	Después	X
Yo	compuesto Q1	2.101	1.430	1.46x
Yo Q3	precio	3.394	1.636	2.07x

Uso	Prompt	Qué hice
Carga masiva	50k/20k/200k generate_series	Acepté + OVERRIDING por 428C9
Índices	plan real, qué nodo ataca	Q1/Q3 sí, Q2 no
Explicación plan	nodo por nodo	Corregí cost, índice, buffers
Specs 4	tablas, filtro activa, orden	Acepté, EXCEPT 0